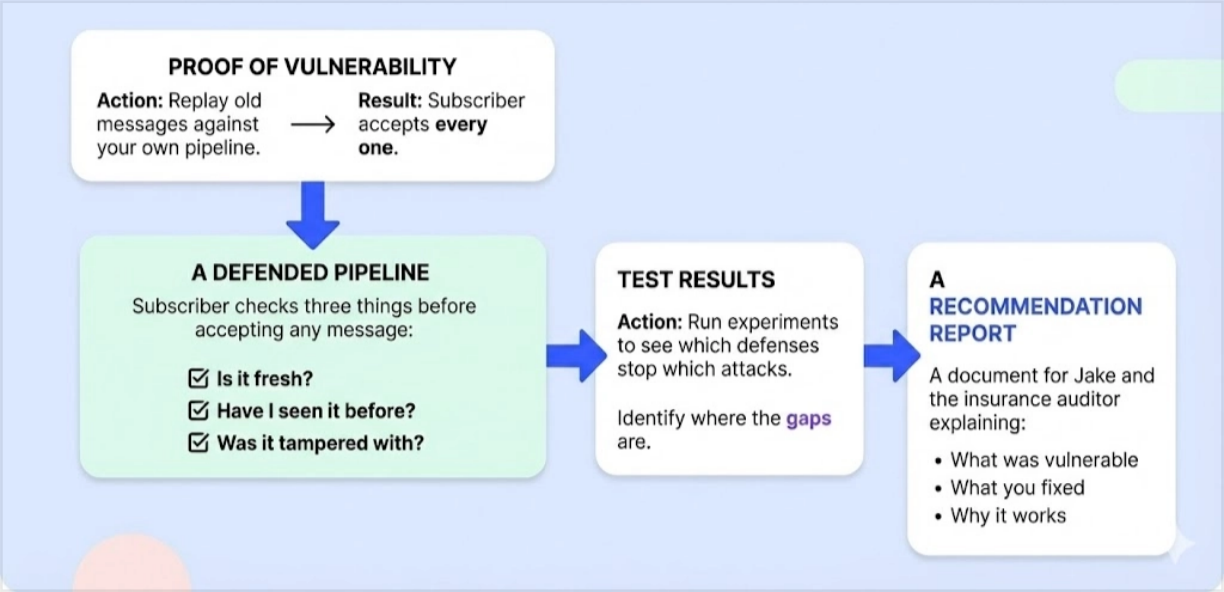
Hydroficient IoT Cyber Defense
Project 6: Defeating Replay Attacks

Step 1: Understanding Replay Attacks

Project Overview



Career note: Replay protection is a real production requirement. IEC 62443 mandates message integrity for industrial control systems. MQTT v5 added Message Expiry Interval to handle stale messages. AWS IoT Core leaves deduplication to the application layer — exactly what you'll build here. After this project, you'll have experiment data and a report to walk an interviewer through, not just a definition.

The Journey

This project follows a structured approach: **prove the vulnerability, build the defenses, test everything, and write the recommendation**.

Step	What You'll Do	Time
Step 1	Understand replay attacks and prove the vulnerability exists in your pipeline	45 min
Step 2	Build three defense mechanisms into your publisher and subscriber	45 min

Step 3	Run controlled experiments and write a professional recommendation report	~100 min

By the time the auditor arrives, your pipeline won't just verify who's connecting — it will verify that every message is fresh, unique, and untampered. And you'll have the test results to prove it.

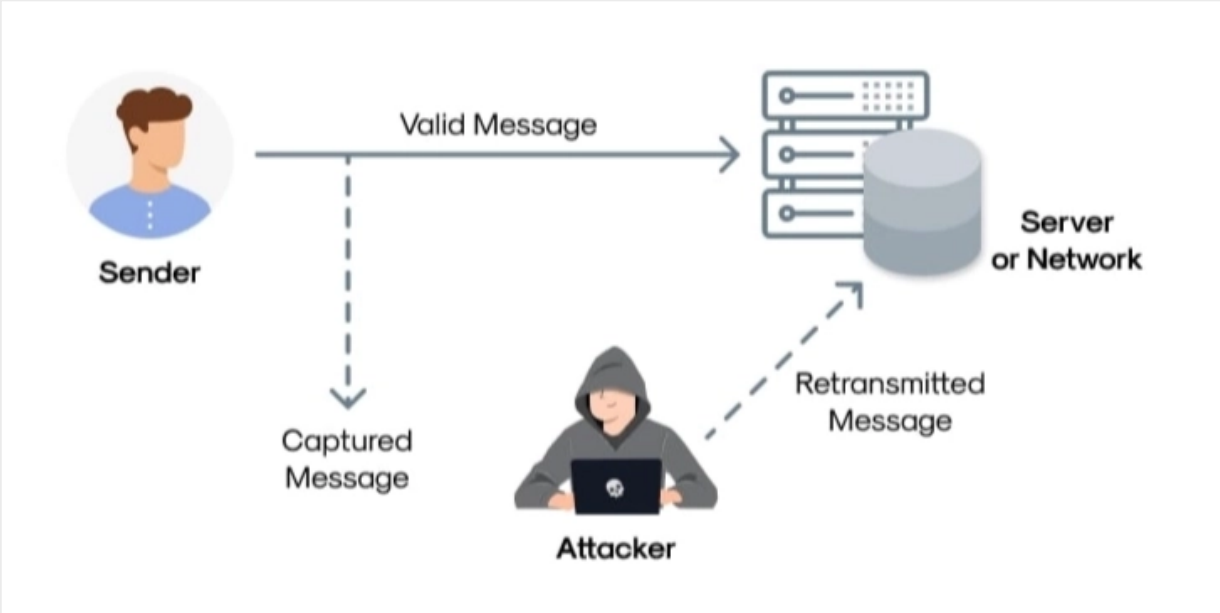
When Hackers Press Rewind

What Is a Replay Attack?

A replay attack is when someone captures a legitimate message and re-sends it later. The message itself is real — it was created by an authorized device, at a real point in time, with valid credentials. The attacker doesn't need to break the encryption or forge a certificate. They just press rewind.

Think about a garage door opener. When you press the button, it sends a radio signal to the receiver. Older systems sent the same signal every time. An attacker could sit in a car with a radio receiver, record the signal, and play it back later to open the garage. The signal was legitimate — it just wasn't *fresh*.

Modern garage doors fixed this with rolling codes. Every button press sends a different code, and the receiver only accepts codes it hasn't seen before. Your MQTT pipeline needs the same kind of protection.



What mTLS Protects Against

Before diving into what goes wrong, it helps to see what your current security stack already handles. Over the last two weeks, you built serious defenses:

Attack	What the Attacker Does	Does mTLS Stop It?

Eavesdropping	Intercepts messages in transit	Yes — TLS encryption makes messages unreadable
Spoofing	Pretends to be a device	Yes — mTLS requires a valid certificate
Man-in-the-middle	Sits between device and broker	Yes — TLS verifies both endpoints
Unauthorized connection	Connects without credentials	Yes — broker requires client certificate
Replay attack	Re-sends a captured legitimate message	**No** — mTLS can't tell old from new

That last row is the problem. Every security measure you've built so far operates at the *connection* level. Once a connection is established and authenticated, the broker delivers every message to subscribers without asking whether the message content is new or old.

Why Replay Attacks Are Dangerous for IoT

Replay attacks are dangerous in IoT because the messages control physical equipment. This isn't someone replaying a login token to read an email. These messages open valves, change pressure, trigger alerts. Let me give you three scenarios. Here are three scenarios at The Grand Marina that show what could go wrong.

Scenario 1: The Phantom Shutoff.

An attacker captures a legitimate maintenance command that temporarily reduces water pressure. They replay it during peak hours, causing low pressure across the hotel while guests are showering. The command looks completely legitimate — it came from an authorized device.

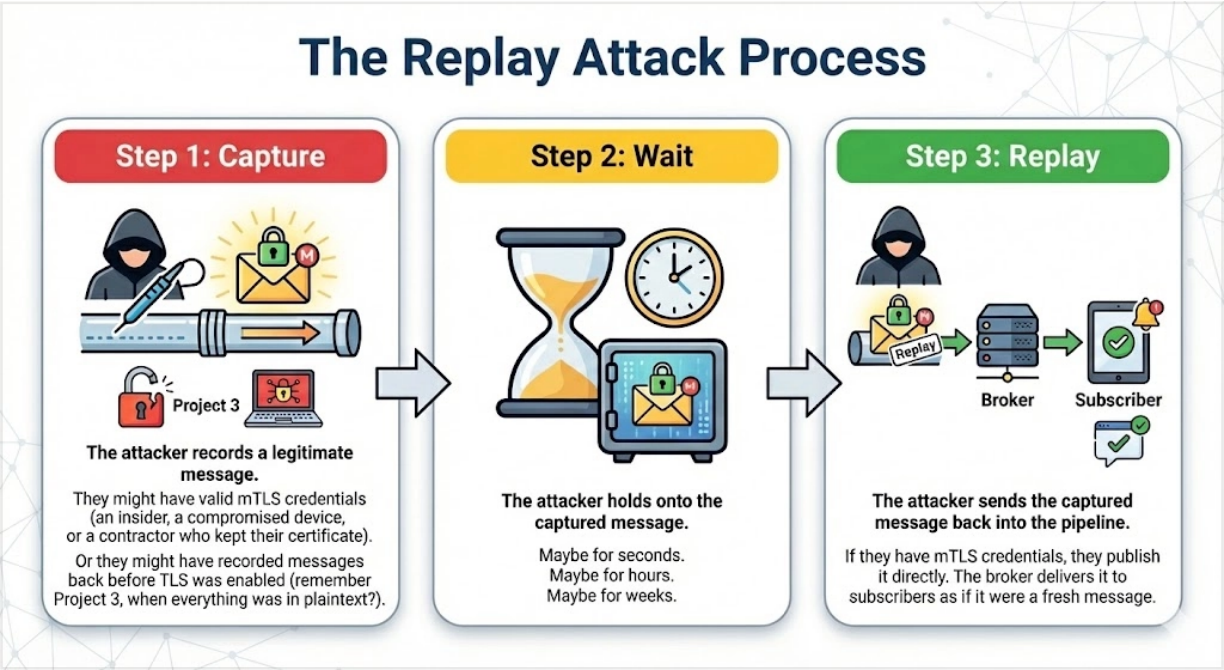
Scenario 2: The Stale Reading

Someone captures a sensor reading showing normal water pressure at 60 PSI. Hours later, a pipe develops a leak and pressure drops to 30 PSI. The attacker replays the old "everything's fine" reading, masking the real problem. The dashboard shows normal conditions while water floods the basement.

Scenario 3: The Repeated Billing Event

HYDROLOGIC devices can trigger billing events when water usage exceeds thresholds. An attacker replays a high-usage reading multiple times, generating false overage charges that show up on The Grand Marina's water bill.

How Replay Attacks Work

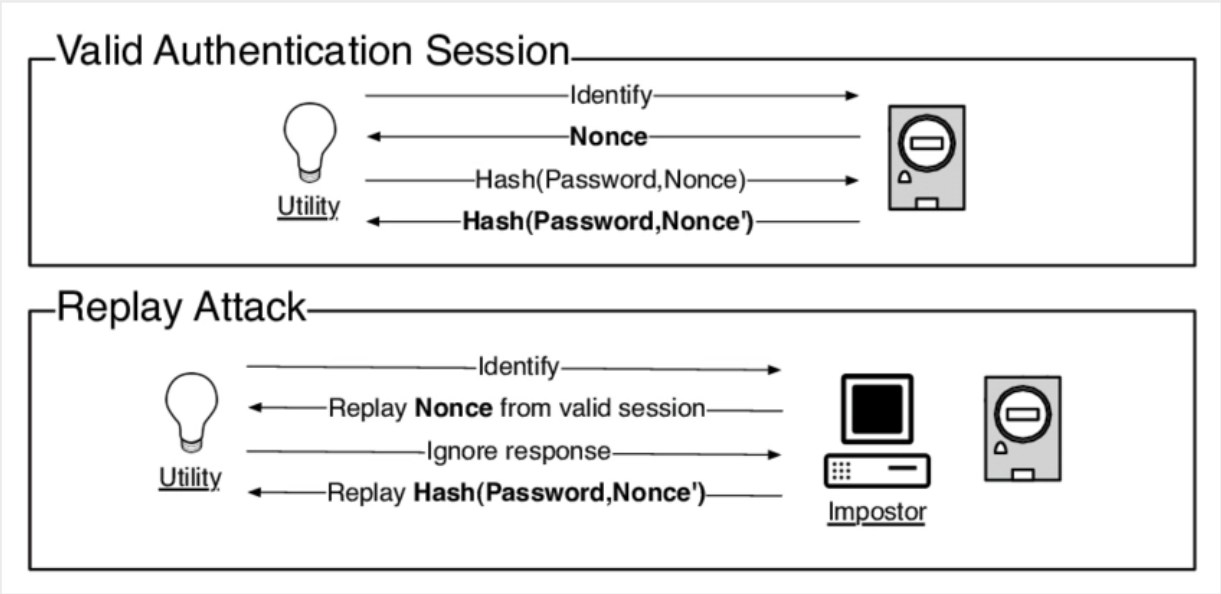


So the subscriber is the problem. It trusts everything. The subscriber parses the JSON and displays the data. It doesn't validate anything about the message content. It doesn't check the timestamp. It doesn't track which messages it's already seen. It doesn't verify that the sender actually computed the payload.

The Gap in Your Defenses

The Trusting Subscriber

You open subscriber_mtls.py on your own screen and scroll down to on_message. You have looked at this function dozens of times over the past two weeks, but you have never read it the way Maya is asking you to read it now, looking for what it does not do. You read through it line by line. It receives a message. It parses the JSON. It prints the values. That is it. There is no validation. No timestamp check. No duplicate detection. No signature verification. Any message that arrives as valid JSON gets displayed as if it were a fresh, legitimate reading. The gap is obvious once you actually look for it.



Three Questions a Subscriber Should Ask

Question 1: "Is this message fresh?"

In a water monitoring system, the freshness of a reading matters. If an attacker replays a reading from 8 hours ago saying "pressure is normal," but a pipe burst two hours ago, the stale reading masks the real emergency. The dashboard shows green while water floods the basement.

When you check into a hotel, your key card has an expiration date. If you try to use it a week after checkout, it doesn't work. Messages should work the same way. Each message has a timestamp, but your current subscriber never looks at it. A message from 8 hours ago gets the same treatment as one from 2 seconds ago.

The fix: compare the message timestamp against the current time. If the message is more than a set number of seconds old (say, 30 seconds), reject it.

Question 2: "Have I seen this before?"

When you enter a concert venue, they scan your ticket. If someone copies your ticket and tries to enter with the duplicate, the scanner rejects it — that ticket number was already used. Messages need the same kind of tracking.

The fix: add a sequence number to each message. The publisher increments this counter with every message. The subscriber tracks the last number it saw from each device. If a message arrives with a number that's the same or lower than the last one seen, it's a replay.

Question 3: "Was this really sent by who it claims?"

A pharmacist doesn't just look at the name on a prescription — they verify the doctor's signature. If someone photocopies a prescription and changes the dosage, the altered signature won't match. Messages need the same integrity

check.

The fix: the publisher computes a signature over the entire message using a shared secret. The subscriber recomputes the signature and compares. If someone modified the message (changed the flow_rate, altered the timestamp, or tampered with any field), the signatures won't match.

Attack Scenario	Timestamp Check	Sequence Counter	Signature (HMAC)
Replayed 60 seconds later	Catches it	Catches it	Passes (message is unchanged)
Replayed within 5 seconds	Misses it (still "fresh")	Catches it	Passes (message is unchanged)
Modified then replayed	Depends on timing	May miss it (new sequence)	Catches it
Replayed with edited timestamp	May miss it	Catches it	Catches it (HMAC covers timestamp)

You study the table row by row. Each defense has a blind spot. Timestamps miss fast replays. Sequence counters can be bypassed by incrementing the number. Signatures do not prevent unmodified replays on their own. But together, they cover each other's weaknesses.

The **date stamp** (timestamp validation) shows when the badge was issued. A badge from last year will not get you past security today — it is expired. But if someone makes a copy the same morning, the date looks fine.	The **unique badge number** (sequence counter) is a one-time number printed on each badge. Security logs every badge number that enters the building. If badge #4721 was already scanned at 9:15 AM, it cannot be used again at 9:16 AM. But if someone creates a brand new badge with a higher number, the counter alone will not catch that.	The **holographic seal** (HMAC signature) is a tamper-evident feature embedded by the badge printer. If someone photocopies a badge, the hologram will not transfer correctly. Any modification to the badge — changing the name, department, or photo — breaks the seal. But an exact, unmodified copy of a recent badge would still have a valid seal.

One feature alone has gaps. All three together make a badge that is extremely

hard to forge, copy, or reuse.

Task: Run a Replay Attack on Your Own Pipeline

Follow the Steps Below

- **Part 1: Run the Normal Pipeline**
  - Start your Project 5 pipeline to confirm everything works.
    - **Terminal 1 — Broker** (if not already running):

```
mosquitto -c mosquitto_mtls.conf -v
```

    - **Terminal 2 — Subscriber:**

```
python subscriber_mtls.py
```

    - **Terminal 3 — Publisher:**

```
python publisher_mtls.py
```

  - Let the publisher send 5-6 messages, then stop it with Ctrl+C. Leave the broker and subscriber running.
  - Your pipeline is confirmed to be working. Now you'll use a second tool to silently capture those same messages.
  -

```
(hydroficient) nefer@Tania-Amandas-MacBook-Pro Project5 % mosquitto -c mosquitto_mtls.conf -v
1773245992: Info: running mosquitto as user: nefer.
1773245992: mosquitto version 2.1.2 starting
1773245992: Config loaded from mosquitto_mtls.conf.
1773245992: Bridge support available.
1773245992: Persistence support available.
1773245992: TLS support available.
1773245992: TLS-PSK support available.
1773245992: Websockets support available.
1773245992: Opening ipv6 listen socket on port 8883.
1773245992: Opening ipv4 listen socket on port 8883.
1773245992: mosquitto version 2.1.2 running
1773245999: New connection from ::1:65307 on port 8883.
1773245999: New client connected from ::1:65307 as GrandMarina-Dashboard (p4, c1, k60, u'HYDROLOGIC-MainBuilding-001').
1773245999: No will message specified.
1773245999: Client GrandMarina-Dashboard negotiated TLSv1.3 cipher TLS_AES_256_GCM_SHA384
1773245999: Sending CONNACK to GrandMarina-Dashboard (0, 0)
1773245999: Received SUBSCRIBE from GrandMarina-Dashboard
1773245999:     hydroficient/grandmarina/# (QoS 1)
1773245999: GrandMarina-Dashboard 1 hydroficient/grandmarina/#
1773245999: Sending SUBACK to GrandMarina-Dashboard
1773246006: New connection from ::1:65406 on port 8883.
1773246006: New client connected from ::1:65406 as HYDROLOGIC-Device-001 (p4, c1, k60, u'HYDROLOGIC-MainBuilding-001').
1773246006: No will message specified.
1773246006: Client HYDROLOGIC-Device-001 negotiated TLSv1.3 cipher TLS_AES_256_GCM_SHA384
1773246006: Sending CONNACK to HYDROLOGIC-Device-001 (0, 0)
1773246007: Received PUBLISH from HYDROLOGIC-Device-001 (d0, q1, r0, m1, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246007: Sending PUBLISH to GrandMarina-Dashboard (d0, q1, r0, m1, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246007: Sending PUBACK to HYDROLOGIC-Device-001 (m1, rc0)
1773246007: Received PUBACK from GrandMarina-Dashboard (Mid: 1, RC:0)
1773246013: Received PUBLISH from HYDROLOGIC-Device-001 (d0, q1, r0, m2, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246013: Sending PUBLISH to GrandMarina-Dashboard (d0, q1, r0, m2, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246013: Sending PUBACK to HYDROLOGIC-Device-001 (m2, rc0)
1773246013: Received PUBACK from GrandMarina-Dashboard (Mid: 2, RC:0)
1773246018: Received PUBLISH from HYDROLOGIC-Device-001 (d0, q1, r0, m3, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246018: Sending PUBLISH to GrandMarina-Dashboard (d0, q1, r0, m3, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246018: Sending PUBACK to HYDROLOGIC-Device-001 (m3, rc0)
1773246018: Received PUBACK from GrandMarina-Dashboard (Mid: 3, RC:0)
1773246023: Received PUBLISH from HYDROLOGIC-Device-001 (d0, q1, r0, m4, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246023: Sending PUBLISH to GrandMarina-Dashboard (d0, q1, r0, m4, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246023: Sending PUBACK to HYDROLOGIC-Device-001 (m4, rc0)
1773246023: Received PUBACK from GrandMarina-Dashboard (Mid: 4, RC:0)
1773246028: Received PUBLISH from HYDROLOGIC-Device-001 (d0, q1, r0, m5, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246028: Sending PUBLISH to GrandMarina-Dashboard (d0, q1, r0, m5, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246028: Sending PUBACK to HYDROLOGIC-Device-001 (m5, rc0)
1773246028: Received PUBACK from GrandMarina-Dashboard (Mid: 5, RC:0)
1773246033: Received PUBLISH from HYDROLOGIC-Device-001 (d0, q1, r0, m6, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246033: Sending PUBLISH to GrandMarina-Dashboard (d0, q1, r0, m6, 'hydroficient/grandmarina/device-001/sensors', ... (283 bytes))
1773246033: Sending PUBACK to HYDROLOGIC-Device-001 (m6, rc0)
1773246033: Received PUBACK from GrandMarina-Dashboard (Mid: 6, RC:0)
1773246035: Received DISCONNECT from HYDROLOGIC-Device-001
1773246035: Client HYDROLOGIC-Device-001 [::1:65406] disconnected.
1773246060: Received PINGREQ from GrandMarina-Dashboard
1773246060: Sending PINGRESP to GrandMarina-Dashboard
```

  -

```
(hydroficient) nefer@Tania-Amandas-MacBook-Pro Project5 % python subscriber_mtls.py
============================================================
Grand Marina Security Dashboard (mTLS)
============================================================
Subscribing to: hydroficient/grandmarina/#
Certificate: certs/device-001.pem
============================================================
/Users/nefer/Desktop/Hydroficient/Project5/subscriber_mtls.py:100: DeprecationWarning: Callback API version 1 is deprecated, update to latest ver
sion
  client = mqtt.Client(client_id=CLIENT_NAME, **MQTT_CLIENT_ARGS)
[TLS] mTLS configured with client certificate

[CONNECTING] localhost:8883...

[LISTENING] Waiting for messages (Ctrl+C to stop)...

[SUCCESS] Connected to broker as GrandMarina-Dashboard
[INFO] Subscribing to: hydroficient/grandmarina/#
[SUBSCRIBED] QoS granted: (1,)
```

```
(hydroficient) nefer@Tania-Amandas-MacBook-Pro Project5 % python publisher_mtls.py
============================================================
HYDROLOGIC Sensor Publisher (mTLS)
============================================================
Device ID: 001
Topic: hydroficient/grandmarina/device-001/sensors
Certificate: certs/device-001.pem
============================================================
/Users/nefer/Desktop/Hydroficient/Project5/publisher_mtls.py:108: DeprecationWarning: Callback API version 1 is depreca
ted, update to latest version
  client = mqtt.Client(client_id=CLIENT_NAME, **MQTT_CLIENT_ARGS)
[TLS] mTLS configured with client certificate

[CONNECTING] localhost:8883...
[SUCCESS] Connected to broker as HYDROLOGIC-Device-001
[INFO] Certificate identity verified by broker

[PUBLISHING] Sending sensor readings (Ctrl+C to stop)...

[1] Published: 53.68 LPM
[PUBLISHED] Message ID: 1
[2] Published: 54.75 LPM
[PUBLISHED] Message ID: 2
[3] Published: 46.99 LPM
[PUBLISHED] Message ID: 3
[4] Published: 46.13 LPM
[PUBLISHED] Message ID: 4
[5] Published: 47.59 LPM
[PUBLISHED] Message ID: 5
[6] Published: 50.45 LPM
[PUBLISHED] Message ID: 6
^C

[INFO] Stopping after 6 messages...
[INFO] Clean disconnect
[INFO] Disconnected from broker
```

- **Part 2: Capture Messages**
  - In **Terminal 4**, run the replay attacker in capture mode. You can find the code file here.

`python replay_attacker.py --mode capture --count 5`

```
(hydroficient) nefer@Tania-Amandas-MacBook-Pro Project5 % python replay_attacker.py --mode capture --count 5
============================================================
REPLAY ATTACK TOOL - Capture Mode
============================================================
Target: 5 messages
Output: captured_messages.json
============================================================
/Users/nefer/Desktop/Hydroficient/Project5/replay_attacker.py:114: DeprecationWarning: Callback API version 1
ated, update to latest version
  client = mqtt.Client(client_id="replay-attacker-capture", **MQTT_CLIENT_ARGS)

[CONNECTING] localhost:8883...
[LISTENING] Waiting for messages to capture...

[CONNECTED] Listening on: hydroficient/grandmarina/#
[CAPTURED 1/5] HYDROLOGIC-Device-001 - 52.47 LPM
[CAPTURED 2/5] HYDROLOGIC-Device-001 - 53.07 LPM
[CAPTURED 3/5] HYDROLOGIC-Device-001 - 54.36 LPM
[CAPTURED 4/5] HYDROLOGIC-Device-001 - 46.77 LPM
[CAPTURED 5/5] HYDROLOGIC-Device-001 - 51.32 LPM

[DONE] Captured 5 messages
[SAVED] captured_messages.json
[SAVED] 5 messages saved to captured_messages.json
```

  - While the capture tool is listening, go back to **Terminal 3** and start the publisher again:

`python publisher_mtls.py`

  - After 5 messages are captured, stop the publisher with Ctrl+C.

```
(hydroficient) nefer@Tania-Amandas-MacBook-Pro Project5 % python publisher_mtls.py
============================================================
HYDROLOGIC Sensor Publisher (mTLS)
============================================================
Device ID: 001
Topic: hydroficient/grandmarina/device-001/sensors
Certificate: certs/device-001.pem
============================================================
/Users/nefer/Desktop/Hydroficient/Project5/publisher_mtls.py:108: DeprecationWarning: Callback API version 1 is depre
ted, update to latest version
  client = mqtt.Client(client_id=CLIENT_NAME, **MQTT_CLIENT_ARGS)
[TLS] mTLS configured with client certificate

[CONNECTING] localhost:8883...
[SUCCESS] Connected to broker as HYDROLOGIC-Device-001
[INFO] Certificate identity verified by broker

[PUBLISHING] Sending sensor readings (Ctrl+C to stop)...
```

Checkpoint	Expected
captured_messages.json exists	Yes -- check your project folder
File contains 5 message entries	Yes -- you can open it to verify
Each entry has a topic, payload, and captured_at field	Yes

With five messages saved to disk, you now have everything you need to simulate the attack. The next step is to replay those captured messages and see how your subscriber responds.

- **Part 3: Replay the Messages**
  - Wait at least 30 seconds after capturing. Then, in **Terminal 4**, replay the captured messages:

```
python replay_attacker.py --mode replay
```

```
(hydroficient) nefer@Tania-Amandas-MacBook-Pro Project5 % python replay_attacker.py --mode replay
============================================================
REPLAY ATTACK TOOL - Replay Mode
============================================================
Loaded 5 captured messages
============================================================
/Users/nefer/Desktop/Hydroficient/Project5/replay_attacker.py:175: DeprecationWarning: Callback API version 1 is de
ated, update to latest version
  client = mqtt.Client(client_id="replay-attacker", **MQTT_CLIENT_ARGS)

[CONNECTING] localhost:8883...

[REPLAYING] Sending captured messages...

[REPLAYING 1/5] Topic: hydroficient/grandmarina/device-001/sensors
  Originally captured: 2026-03-11T16:23:33.057308Z
  Replayed at: 2026-03-11T16:26:54.339257Z

[REPLAYING 2/5] Topic: hydroficient/grandmarina/device-001/sensors
  Originally captured: 2026-03-11T16:23:38.061074Z
  Replayed at: 2026-03-11T16:26:55.344921Z

[REPLAYING 3/5] Topic: hydroficient/grandmarina/device-001/sensors
  Originally captured: 2026-03-11T16:23:43.067147Z
  Replayed at: 2026-03-11T16:26:56.346746Z

[REPLAYING 4/5] Topic: hydroficient/grandmarina/device-001/sensors
  Originally captured: 2026-03-11T16:23:48.071642Z
  Replayed at: 2026-03-11T16:26:57.352822Z

[REPLAYING 5/5] Topic: hydroficient/grandmarina/device-001/sensors
  Originally captured: 2026-03-11T16:23:53.073310Z
  Replayed at: 2026-03-11T16:26:58.357161Z

[DONE] Replayed 5 messages
[NOTE] Check your subscriber - did it accept them all?
```

  - Now check **Terminal 2** (the subscriber). You should see that all 5 replayed messages were accepted and displayed as normal readings.

```
==================================================
[RECEIVED] Topic: hydroficient/grandmarina/device-001/sensors
[TIME] 11:26:54
--------------------------------------------------
  Device: HYDROLOGIC-Device-001
  Flow Rate: 52.47 LPM
  Pressure (Up): 61.48 PSI
  Pressure (Down): 56.16 PSI
  Status: operational

==================================================
[RECEIVED] Topic: hydroficient/grandmarina/device-001/sensors
[TIME] 11:26:55
--------------------------------------------------
  Device: HYDROLOGIC-Device-001
  Flow Rate: 53.07 LPM
  Pressure (Up): 58.07 PSI
  Pressure (Down): 57.59 PSI
  Status: operational

==================================================
[RECEIVED] Topic: hydroficient/grandmarina/device-001/sensors
[TIME] 11:26:56
```

What You've Built
You've proven the vulnerability exists. Your mTLS subscriber accepted every replayed message without any warning — it couldn't tell the difference between a fresh reading and a recording from a minute ago. This is the gap you'll close in Step 2 by adding timestamp validation, sequence counters, and HMAC signing.

Once done, answer the following questions:
1. How many messages were captured?
Answer: 5 messages were captured.

2. How many replayed messages were accepted?
Answer: All 5 replayed messages were accepted by the subscriber.

3. Did the subscriber display any warning?
Answer: No, the subscriber did not display any warning.

4. Were the replayed readings identical to the originals?
Answer: Yes, the replayed readings were identical to the original messages.

5. Could you tell the difference between real and replayed messages?
Answer: No, there was no visible difference between the real messages and the replayed messages.

Step 2: Building Defenses
Understanding the Three Lines of Defense

Defense 1: Timestamp Validation
Your publisher already puts a timestamp in every message — has since week 3 (Project 3). The problem is nobody checks it.
Timestamp validation applies one rule: if a message is older than a configured window (say, 30 seconds), reject it.
What it catches: Replays that happen after the window expires (the most common type).
What it misses: Immediate replays (within the window). If an attacker replays a message 3 seconds after capturing it, the timestamp still looks fresh.

Defense 2: Sequence Counter
A sequence counter assigns each message a unique, incrementing number — like a concert ticket with a unique barcode. Once it's scanned, it can't be used again. The subscriber tracks the last sequence number it saw from each device and rejects any message with a number that's the same or lower. Once barcode #847 is scanned at the gate, it won't work again — even if someone presents a perfect photocopy five seconds later.

What it catches: Any exact replay (the sequence number was already used) and most replay attacks regardless of timing.
What it misses: Modified replays where the attacker increments the sequence number. If the attacker knows the current counter is at 42, they could set their replayed message to sequence 43. The counter alone won't catch that.

Defense 3: HMAC Message Signing
The third defense protects the message itself — not when it was sent, not whether you've seen it, but whether anyone tampered with it.
Think of a wax seal on a letter. In the days before envelopes, important letters were folded and sealed with wax stamped by the sender's signet ring. If the seal was intact when the letter arrived, you knew nobody had tampered with it. If the seal was broken, you knew someone had opened and possibly altered the letter.
HMAC (Hash-based Message Authentication Code) works the same way. The publisher computes a signature over the entire message contents using a secret key, and the subscriber verifies it. If anyone modifies even a single character of the message, the signature becomes invalid.
HMAC works differently from TLS encryption. TLS protects the message *in transit* — nobody can read it while it's traveling between devices. HMAC protects the message *content* — nobody can modify it after the publisher created it. TLS is like an armored truck. HMAC is like a tamper-evident seal on the package inside the truck.

The function excludes the hmac field from the computation. This is important — you can't include the signature in the data you're signing, because you don't know the signature yet when you compute it. The publisher computes the HMAC over everything *except* the hmac field, then attaches the result.

The sort_keys=True parameter ensures that the JSON string is always in the same order. Without this, {"a": 1, "b": 2} and {"b": 2, "a": 1} would produce different signatures even though they represent the same data. Sorting the keys guarantees consistency.

The SHARED_SECRET is a key that both the publisher and subscriber know. In production, this would be securely provisioned to each device (similar to how certificates are provisioned). For this project, both scripts use the same hard-coded string.

What it catches: Any message modification — changed sensor values, altered timestamps, edited device IDs. The signature covers every field.
What it misses: Exact replays of unmodified messages. If the attacker replays the message byte-for-byte without changing anything, the HMAC is still valid. That's why you need timestamps and sequence counters to catch unmodified replays.

Why All Three Together
The final page shows a matrix:

Attack	Timestamp	Counter	HMAC	All Three
Replay after 60 seconds	Blocked	Blocked	Passes	Blocked
Replay within 5 seconds	Passes	Blocked	Passes	Blocked
Modified with new data	Depends	May pass	Blocked	Blocked
Modified with future sequence	Depends	Passes	Blocked	Blocked

The last column is the point. With all three defenses active, every type of replay attack is blocked. Timestamps catch old replays. Counters catch recent replays. HMAC catches tampered replays. Together, they leave no gaps.

The Defended Publisher

The defended publisher (publisher_defended.py) is based on the publisher_mtls.py you built last week. The mTLS connection setup is identical — same certificates, same broker configuration. The changes are all in how the message is constructed.

Two new fields: sequence (an incrementing integer) and hmac (a hex string). Everything else stays the same. The sensor readings, device ID, timestamp format, and topic structure are unchanged.

Now let's look at how these fields are generated, starting from the top of the file.

New Imports
The defended publisher adds two imports at the top of the file:

```
import hmac
import hashlib
```

Both hmac and hashlib are part of Python's standard library. You don't need to pip install anything. They've been available in every Python version since 2.x.
hmac provides the HMAC algorithm — it combines a secret key with a message to produce a signature. hashlib provides the hashing function (SHA-256) that HMAC uses internally.
Right below the imports is a new constant that was not in the Week 5 (Project 5) version.

The Shared Secret

Near the top of the file, you'll find a constant:

```
SHARED_SECRET = "grandmarina-hydroficient-2024-secret-key"
```

This is the key used to compute the HMAC signature. Both the publisher and subscriber must use exactly the same secret. If they don't match, every message will fail HMAC validation.

In a production system, this secret would be securely provisioned to each device — similar to how certificates are distributed. For this project, both scripts use the same hard-coded string so you can focus on understanding the mechanism rather than managing key distribution.

The secret handles authenticity. The next piece handles ordering.

The Sequence Counter

The publisher tracks a global counter that starts at zero and increments with each message:

```
sequence_counter = 0

def generate_defended_reading():
    global sequence_counter
    sequence_counter += 1

    message = {
        "device_id": f"HYDROLOGIC-Device-{DEVICE_ID}",
        "timestamp":
datetime.now(timezone.utc).isoformat().replace("+00:00", "Z"),
        "sequence": sequence_counter,
        # ... sensor readings ...
    }
```

The **global** keyword is necessary because the counter lives outside the function but gets modified inside it. Each time the publisher sends a message, the counter goes up by one. Message 1 gets sequence: 1, message 2 gets sequence: 2, and so on.

If the publisher restarts, the counter resets to zero. In a production system, the counter would be stored persistently (in a file or database) so it survives restarts. For this project, a fresh counter on each restart is fine because the subscriber also resets its tracking when it restarts.

The sequence counter handles ordering. The HMAC computation handles authenticity.

The HMAC Computation

The compute_hmac() **function creates a signature over the entire message:**

```
def compute_hmac(message_dict):
    # Step 1: Copy message, exclude the hmac field
    msg_copy = {k: v for k, v in message_dict.items() if k !=
"hmac"}
```

```
    # Step 2: Convert to a consistent string
    msg_string = json.dumps(msg_copy, sort_keys=True)

    # Step 3: Sign with shared secret using SHA-256
    signature = hmac.new(
        SHARED_SECRET.encode("utf-8"),
        msg_string.encode("utf-8"),
        hashlib.sha256
    ).hexdigest()

    return signature
```

Step 1 creates a copy of the message dictionary without the hmac field. You can't include the signature in the data you're signing — the signature doesn't exist yet when you start computing it. The dictionary comprehension {k: v for k, v in message_dict.items() if k != "hmac"} creates a new dictionary containing every key-value pair except hmac.

Step 2 converts the dictionary to a JSON string with sorted keys.
The sort_keys=True parameter guarantees that the fields always appear in alphabetical order. Without this, Python dictionaries might produce {"device_id": ..., "readings": ...} one time and {"readings": ..., "device_id": ...} another time. Different orderings would produce different signatures, causing validation to fail on the subscriber end.

Step 3 computes the actual HMAC-SHA256. hmac.new() takes three arguments: the secret key (as bytes), the message (as bytes), and the hash algorithm.
The .hexdigest() call returns the signature as a readable hex string like "a3f2b8c91d4e..." rather than raw bytes.
The sort_keys=True detail is easy to overlook — without it, a signature mismatch caused by different key ordering could take hours to debug.

How the Message Is Assembled
The **generate_defended_reading()** function puts it all together. First, the message is built with all its fields (including sequence and timestamp). Then the HMAC is computed over the complete message. Finally, the HMAC is added to the message. The subscriber will reverse this process — remove the HMAC, compute it fresh, and compare.

That covers the complete file — imports, secret, counter, HMAC function, and message assembly.

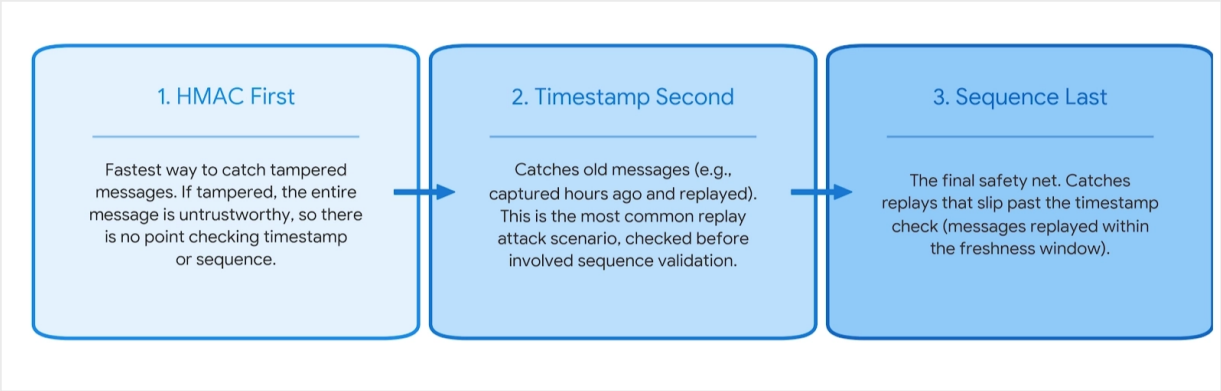
What the Output Looks Like
Each line shows the flow rate, the sequence number, and the first 12 characters of

the HMAC. The full HMAC is a 64-character hex string, but the output truncates it for readability.
Notice that the sequence number increases with every message and the HMAC is different each time. Even though the message structure is the same, the changing timestamp and sequence number produce a unique signature for every message.

The Defended Subscriber

The defended subscriber (subscriber_defended.py) is based on the subscriber_mtls.py you built last week. The structure is the same as Week 5 (Project 5) — the subscriber connects to the broker over mTLS, subscribes to the wildcard topic, and processes messages in the on_message callback. The difference is what happens inside that callback.

The Validation Pipeline
The validate_message() function runs all three checks in a specific order: HMAC first, then timestamp, then sequence. The order is not arbitrary. If any check fails, the function returns immediately with False and the results so far. The subscriber does not need to know why the other checks would have passed or failed — one failure is enough to reject.



HMAC Verification
The verify_hmac() function reverses what the publisher did.
The process mirrors the publisher exactly: remove the hmac field, serialize with sorted keys, compute the HMAC using the same shared secret. If the result matches the hmac field in the message, the message is authentic. If not, something changed after the publisher created it.

One thing to note: the comparison uses hmac.compare_digest() instead of ==. This is a security measure — more on why below.
Messages from the old publisher_mtls.py (which do not have an hmac field) will be caught by the if received_hmac is None check. This means the defended subscriber is backward-incompatible by design — it requires the defended publisher.

The HMAC comparison uses compare_digest() instead of ==. Why not just use ==?
Regular == stops at the first mismatch, so it takes slightly longer to reject strings that match on early characters. An attacker can measure those tiny timing differences and gradually figure out the correct signature. compare_digest() always takes the same amount of time. Same result, no information leakage. You have the full picture now — publisher signs, subscriber verifies. Three checks, specific order, fail-fast on any mismatch. Next step is running it.

Timestamp Freshness Check
The MAX_AGE_SECONDS constant at the top of the file controls the freshness window. At 30 seconds, any message with a timestamp more than 30 seconds old gets rejected. You can adjust this value — smaller is more secure but less tolerant of network delays.
The function parses the ISO-format timestamp from the message, calculates how many seconds old it is, and compares against the window. It also returns the age so the subscriber can display it (useful for understanding why a message was accepted or rejected).
The .replace("Z", "+00:00") call converts the UTC indicator format. The publisher writes timestamps ending in Z (e.g., 2025-02-12T14:30:00Z), and Python's fromisoformat() needs +00:00 instead.

Sequence Counter Tracking
The device_counters dictionary maps each device ID to the highest sequence number the subscriber has seen from that device. When a new message arrives, the subscriber checks whether its sequence number is higher than the last one recorded.
The dictionary is keyed by device ID because The Grand Marina has multiple devices. Device-001 might be on sequence 47 while Device-002 is on sequence 12. Each device has its own independent counter.
When the subscriber restarts, device_counters resets to an empty dictionary. The first message from each device after a restart will always be accepted (because 0 is the default, and any positive sequence is greater than 0). In a production system, you would persist these counters to disk, but for this project, the reset-on-restart behavior is acceptable.

The on_message Callback
Accepted messages show [ACCEPTED] with all three checks passing. Rejected messages show [REJECTED] with the specific check that failed and why. The subscriber also tracks running statistics so you can see the accept/reject ratio over time.

Expected Output

When running with the defended publisher, you will see accepted messages:

```
[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 51.23 LPM |
Seq: 1
  HMAC: PASS | Timestamp: PASS (Age: 0.3s (max: 30s)) |
Sequence: PASS
  Stats: 1 accepted, 0 rejected (1 total)

[ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 48.76 LPM |
Seq: 2
  HMAC: PASS | Timestamp: PASS (Age: 0.2s (max: 30s)) |
Sequence: PASS
  Stats: 2 accepted, 0 rejected (2 total)
```

When the replay attacker sends captured messages, you will see rejections:

```
[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 51.23 LPM |
Seq: 1
  Failed check: SEQUENCE
  Reason: Sequence 1 <= last seen 5 (replay detected)
  Stats: 5 accepted, 1 rejected (6 total)
```

Task: Secure Your Pipeline Against Replay

Follow the Steps Below

- Part 1: Run the Defended Pipeline
    - **Terminal 1 — Broker** (if not already running):

```
mosquitto -c mosquitto_mtls.conf -v
```

    - **Terminal 2 — Defended Subscriber:**

```
python subscriber_defended.py
```

    - **Terminal 3 — Defended Publisher:**

```
python publisher_defended.py
```

    - **What you should see** in the subscriber (Terminal 2):
        - [ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 51.23 LPM | Seq: 1
        - HMAC: PASS | Timestamp: PASS (Age: 0.3s (max: 30s)) | Sequence: PASS
        - Stats: 1 accepted, 0 rejected (1 total)

        - [ACCEPTED] Device: HYDROLOGIC-Device-001 | Flow: 48.76 LPM | Seq: 2
        - HMAC: PASS | Timestamp: PASS (Age: 0.2s (max: 30s)) | Sequence: PASS
        - Stats: 2 accepted, 0 rejected (2 total)
    - All three checks pass: HMAC is valid, timestamp is fresh, sequence is new. Let the publisher send 5 messages, then stop it with Ctrl+C.

Leave the broker and subscriber running.
    - **Take a screenshot** of the subscriber showing [ACCEPTED] messages with all three checks passing.

Checkpoint	Expected
Publisher shows seq= numbers incrementing	Yes
Publisher shows hmac= values (different each time)	Yes
Subscriber shows [ACCEPTED] for every message	Yes
All three checks show PASS	Yes

- **Part 2: Try the Replay Attack**
    - Now capture messages from the defended publisher and try to replay them.
    - **Terminal 4 — Capture messages:** Start the publisher again in Terminal 3, then capture 5 messages:

```
python replay_attacker.py --mode capture --count 5
```

    - After capturing 5 messages, stop the publisher. Wait 30 seconds (so the timestamps expire), then replay:

```
python replay_attacker.py --mode replay
```

    - **What you should see** in the subscriber (Terminal 2):

```
[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 51.23 LPM |
Seq: 1
Failed check: SEQUENCE
Reason: Sequence 1 <= last seen 5 (replay detected)
Stats: 5 accepted, 1 rejected (6 total)
[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 48.76 LPM |
Seq: 2
Failed check: SEQUENCE
Reason: Sequence 2 <= last seen 5 (replay detected)
Stats: 5 accepted, 2 rejected (7 total)
```

    - The subscriber rejects every replayed message. The sequence counter catches them — the subscriber already saw sequences 1-5, so it won't accept them again.
    - Take a screenshot of the subscriber showing [REJECTED] messages with the failure reasons.

- **Part 3: Quick Replay**
    - What if the attacker replays immediately — before the timestamp window expires?
    - Restart the subscriber (so counters reset), then start the publisher

again. While the publisher is sending, quickly capture and replay:

```
python replay_attacker.py --mode capture --count 3
```

- Then immediately (within 10 seconds) replay:

```
python replay_attacker.py --mode replay
```

- What you should see: The subscriber still rejects the replays. Even though the timestamps are fresh (within 30 seconds), the sequence counter catches the duplicates:

```
[REJECTED] Device: HYDROLOGIC-Device-001 | Flow: 52.41 LPM |
Seq: 1
  Failed check: SEQUENCE
  Reason: Sequence 1 <= last seen 3 (replay detected)
```

- This is why you need both defenses. The timestamp alone would have let these through. The sequence counter catches what timestamps miss.
- Make sure you have answer to this table:

Scenario	HMAC Check	Timestamp Check	Sequence Check	Result
Fresh message from defended publisher	PASS	PASS	PASS	ACCEPTED
Replay after 30+ seconds	PASS	FAIL	FAIL	REJECTED
Replay within 10 seconds	PASS	PASS	FAIL	REJECTED

- For each check, write PASS or FAIL. For the result, write ACCEPTED or REJECTED.

Step 3: Attack and Defense Lab

Designing Your Experiments

You proved the defended pipeline rejects replays. Good. But here is the problem: you ran three tests by hand. The auditor is going to ask how the system performs against twelve different attack scenarios. "It works, trust me" does not go in a compliance report.

We need data. Twelve experiments. Four defense configurations, three attack

types. Each one tells you something different about where the gaps are — and why all three defenses matter.

Back to the Scientific Method
In Week 4 (Project 4), you ran TLS benchmark experiments following a structured process: form a question, state a hypothesis, run the test, observe the results, draw a conclusion. That same discipline applies here. You're not just clicking buttons — you're designing experiments that produce evidence about which defenses work against which attacks.

The difference is scope. In Week 4 (Project 4), you tested one variable (TLS on vs. off) with one metric (latency). Here, you're testing four defense configurations against three attack types. That's twelve experiments, and each one tells you something different about your security posture.

What Changes and What Stays the Same
To recommend the right defense to Marcus, you need to test them fairly. That means changing one thing at a time (the defense configuration) while keeping everything else the same (the broker, certificates, message count). This is how you isolate what's actually working — and produce evidence that holds up under scrutiny.

Independent variable (what you change): The defense configuration on the subscriber. You'll test four settings:

Configuration	Defenses Active	What It Simulates
none	No validation	Your original subscriber from Week 5 (Project 5)
timestamp	Timestamp check only	A subscriber that only checks message freshness
counter	Sequence counter only	A subscriber that only tracks message numbers
all	Timestamp + counter + HMAC	The fully defended subscriber

Dependent variable (what you measure): The rejection rate — what percentage of replay attack messages does the subscriber reject? A rejection rate of 0% means the attack succeeded completely. A rejection rate of 100% means the defense blocked every attempt.
Controlled variables (what stays the same): The broker configuration, the mTLS certificates, the number of test messages (5 per experiment), and the message format. Only the defense settings and attack types change between experiments.

The Attack Types

You'll test three types of replay attack, each targeting a different weakness:

Immediate replay sends captured messages within 5 seconds of the original. This tests whether defenses can catch replays that are still "fresh" — the timestamp hasn't expired yet. This is the trickiest attack to defend against because the message looks recent.	**Delayed replay** sends captured messages after 60 seconds. This simulates the most common real-world scenario — an attacker records messages and replays them minutes or hours later. Timestamp validation should catch this easily.	**Modified replay** sends captured messages with altered sensor values (specifically, setting `flow_rate` to 0.0 to simulate a shutoff command). This tests whether defenses can detect tampering. The attacker isn't just replaying — they're changing the data to cause specific harm.

Knowing what the attacks do is one thing. Predicting what happens when each defense meets each attack is another — and that is the core of the scientific method.

Your Hypotheses

Before running the experiments, think about what you expect to happen. Write your hypotheses down before running the experiments. Part of the scientific method is comparing your predictions against reality.

Experiment 1 (no defense): With no validation, every attack type should succeed. The rejection rate should be 0% across the board. All messages are accepted regardless of age, sequence, or integrity.

Experiment 2 (timestamp only): Delayed replays should be caught (they're older than 30 seconds). Immediate replays should slip through (they're still within the window). Modified replays depend on timing — the timestamp might be valid even if the data is changed.

Experiment 3 (counter only): Immediate and delayed replays should be caught (they reuse old sequence numbers). Modified replays are trickier — if the attacker sets a higher sequence number, the counter won't catch it.

Experiment 4 (all defenses): Everything should be caught. 100% rejection rate across all attack types. Timestamps catch old replays, counters catch recent ones, and HMAC catches modified ones.

The defense_tester.py Tool

The defense_tester.py script automates the experiments. Instead of manually starting publishers, subscribers, and attackers in separate terminals, it simulates

the entire sequence in a single process.
The script works in two phases for each experiment. First, it generates legitimate messages and processes them through the subscriber's validation logic (to establish the baseline counters). Then it creates attack messages and runs them through the same validation. The rejection rate tells you how effective the defense was.

To run a single experiment:
```
python defense_tester.py --defense none --attack all
```
This tests the "no defense" configuration against all three attack types.

To run all four defense configurations against all attacks:
```
python defense_tester.py --defense none --attack all
python defense_tester.py --defense timestamp --attack all
python defense_tester.py --defense counter --attack all
python defense_tester.py --defense all --attack all
```
Each command produces detailed output showing every message, whether it was accepted or rejected, and why. At the end, it prints a summary table and saves the results to experiment_results.json.

Reading the Results
For each experiment, the output follows a consistent pattern. The four steps mirror the scientific method: generate data (setup), establish the baseline (control), create the attack (independent variable), and measure the outcome (dependent variable). After running all four defense configurations, the script prints a summary table. The four steps mirror the scientific method: generate data (setup), establish the baseline (control), create the attack (independent variable), and measure the outcome (dependent variable). After running all four defense configurations, the script prints a summary table:

Generating the Chart
After running all experiments, generate a visualization:
```
python defense_tester.py --mode chart
```
This creates defense_comparison.png — a grouped bar chart showing the rejection rate for each defense configuration and attack type. The chart makes patterns immediately visible: no defense blocks nothing, individual defenses have gaps, and all three together achieve 100% rejection.

The chart requires matplotlib. If you haven't installed it yet:
```
pip install matplotlib
```
This is the only new package for Project 6. Everything else (hmac, hashlib, json, argparse) is built into Python

Task: Run the Defense Experiments (Mandatory)

Follow the Steps Below

- **Write Your Hypotheses**
  - Before running any experiments, predict what you think will happen. For each experiment, write down the expected rejection rate for each attack type in the Google Doc. You'll compare these predictions against your actual results.

Experiment	Immediate Replay	Delayed Replay	Modified Replay
No defense	0%	0%	0%
Timestamp only	0%	100%	0%
Counter only	100%	100%	0%
All defenses	100%	100%	100%

- **Experiment 1: No Defense (Baseline)**
  - This establishes what happens with zero message validation — your original Project 5 subscriber.
    python defense_tester.py --defense none --attack all

  - Record your results:

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	5	0	0%
Delayed replay	5	0	0%
Modified replay	5	0	0%

- **Expected result:** 0% rejection across all attack types. No defense means every message gets through.
  - Since no validation is active, the subscriber accepts every message. It does not check timestamps, sequence numbers, or message integrity, so all replayed and modified messages pass through with a **0% rejection rate**.

- **Experiment 2: Timestamp Only**

```
python defense_tester.py --defense timestamp --attack all
```

  - Record your results:

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	5	0	0%
Delayed replay	0	5	100%
Modified replay	5	0	0%

  - **What to notice:** Delayed replays should be caught (they're older than 30 seconds). Immediate replays should get through (they're still

within the window). What about modified replays?
    - **Answer**: With only the timestamp defense enabled, the subscriber rejects messages that are older than the 30-second window. This means delayed replay attacks are blocked. However, immediate replay attacks and modified replay attacks still pass because their timestamps are still fresh.

- **Experiment 3: Counter Only**

```
python defense_tester.py --defense counter --attack all
```

    - Record your results:

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	0	5	100%
Delayed replay	0	5	100%
Modified replay	5	0	0%

    - **What to notice:** The counter should catch immediate and delayed replays (they reuse old sequence numbers). Modified replays are assigned new sequence numbers by the attacker — does the counter catch those?
    - **Explanation**: The counter defense tracks the last sequence number seen from the device. Immediate and delayed replay attacks reuse old sequence numbers (1–5), so the subscriber rejects them. However, modified replay attacks use new sequence numbers (6–10), so the counter alone cannot detect them and those messages are accepted.

- **Experiment 4: All Defenses**

```
python defense_tester.py --defense all --attack all
```

    - Record your results:

Attack Type	Accepted	Rejected	Rejection Rate
Immediate replay	0	5	100%
Delayed replay	0	5	100%
Modified replay	0	5	100%

    - **Expected result:** 100% rejection across all attack types. This is the configuration you'll recommend for The Grand Marina.
    - **Explanation**: When all defenses are enabled (HMAC, timestamp, and sequence counter), every attack is blocked. Immediate and delayed replays are rejected because their sequence numbers were already seen or their timestamps are too old. Modified replays are rejected because the HMAC check detects that the message payload was changed.

- **Compile the Summary Table**
  - Compile all your results into a single table:

Defense	Immediate Replay	Delayed Replay	Modified Replay
No defense	0%	0%	0%
Timestamp only	0%	100%	0%
Counter only	100%	100%	0%
All defenses	100%	100%	100%

  - Explanation: When all defenses are enabled (HMAC, timestamp, and sequence counter), every attack is blocked. Immediate and delayed replays are rejected because their sequence numbers were already seen or their timestamps are too old. Modified replays are rejected because the HMAC check detects that the message payload was changed.

- **Generate the Chart**
  - After running all four experiments, generate the visualization: python defense_tester.py --mode chart

  - **What you should see:**

```
[SAVED] Chart saved to defense_comparison.png
[INFO] Open defense_comparison.png to view the comparison
```

Open defense_comparison.png. The chart shows a grouped bar chart with four defense configurations on the x-axis and rejection rate on the y-axis. Three colored bars per configuration represent the three attack types.
Take a screenshot of both your terminal output (showing the experiment summary) and the generated chart.

Checkpoint	Expected
experiment_results.json exists	Yes — contains all 12 experiment results
defense_comparison.png exists	Yes — the grouped bar chart
Chart shows four groups of three bars	Yes
"All Three Defenses" group shows 100% on all bars	Yes

- **Compare Hypotheses Against Results**
  1. **Which predictions were correct?** (Most people get the "no defense" and "all defenses" cases right.)
  2. **Which results surprised you?** Were there any attack/defense

combinations you expected to block that didn't, or vice versa?
    3. **Why did individual defenses have gaps?** Think about what each defense checks and what it doesn't.
- **Submit Deliverables**
  - Submit the following to the **shared Google Doc**:
    1. **Hypothesis table** (your predictions before running experiments)
    2. **Summary table** (actual results from all 12 experiments)
    3. **Screenshot** of the defense_comparison.png chart
    4. **Screenshot** of terminal output showing the experiment summary
    5. **Hypothesis comparison answers** (3 questions above)

Choosing the Right Defense

The Trade-Offs

Across the top: **Effectiveness**, **Complexity**, **Performance**, **Failure Mode**.

Defense	Effectiveness	Code Complexity	Performance Impact	Failure Mode
Timestamp	Partial — misses fast replays	Low — one comparison	Minimal — a single subtraction	Clock drift between devices can cause false rejections
Sequence Counter	High — catches all exact replays	Medium — must persist state	Minimal — one dictionary lookup	Counter reset on restart accepts first replay; requires per-device tracking
HMAC	High — catches all tampering	Medium — cryptographic computation	Low — SHA-256 is fast (~microseconds)	Secret management; if secret is compromised, all devices are affected

All Three	Complete	Higher — three checks per message	Still low — all operations are fast	More potential failure points, but defense-in-depth means one failure doesn't compromise everything

She taps the "All Three" row.

Performance in Perspective

Timestamp validation: Parsing a date and subtracting two times. Microseconds — fractions of a millisecond. Your HYDROLOGIC devices publish every 5 seconds. A check that takes 0.001 seconds against a 5-second interval is negligible.
Sequence counter: A dictionary lookup. Python dictionaries are hash tables — O(1), constant time regardless of how many devices you're tracking.
HMAC-SHA256: Serialize the message to JSON, run the hash. For messages your size — a few hundred bytes — single-digit microseconds. Same algorithm that secures every HTTPS website on the internet.

Added together, all three checks take less than a millisecond per message. Compare that to the network round-trip time for the MQTT message — typically 5 to 50 milliseconds on a local network — and the 5-second interval between readings. The validation overhead is invisible. When Marcus asks, tell him: less than a millisecond, zero impact on operations.

Industry Context

Insurance auditors look for alignment with industry standards. Here's what they'll be checking against — and the good news is, we align with all of them.

- AWS IoT Core uses message deduplication to prevent replays. When a device publishes with QoS 1, AWS tracks message IDs and discards duplicates within a 5-minute window. This is conceptually similar to your sequence counter, though implemented at the broker level rather than the subscriber level.
- IEC 62443 is the security standard for industrial control systems — the kind of systems that run water treatment plants. It requires message integrity checks and replay protection. In other words, what you built this week is what the standard says you should build.
- **MQTT v5** added a built-in message expiry feature. Publishers set how long a message is valid, and the broker throws out anything older. Same idea as your timestamp check, but handled by the broker instead of your

code.

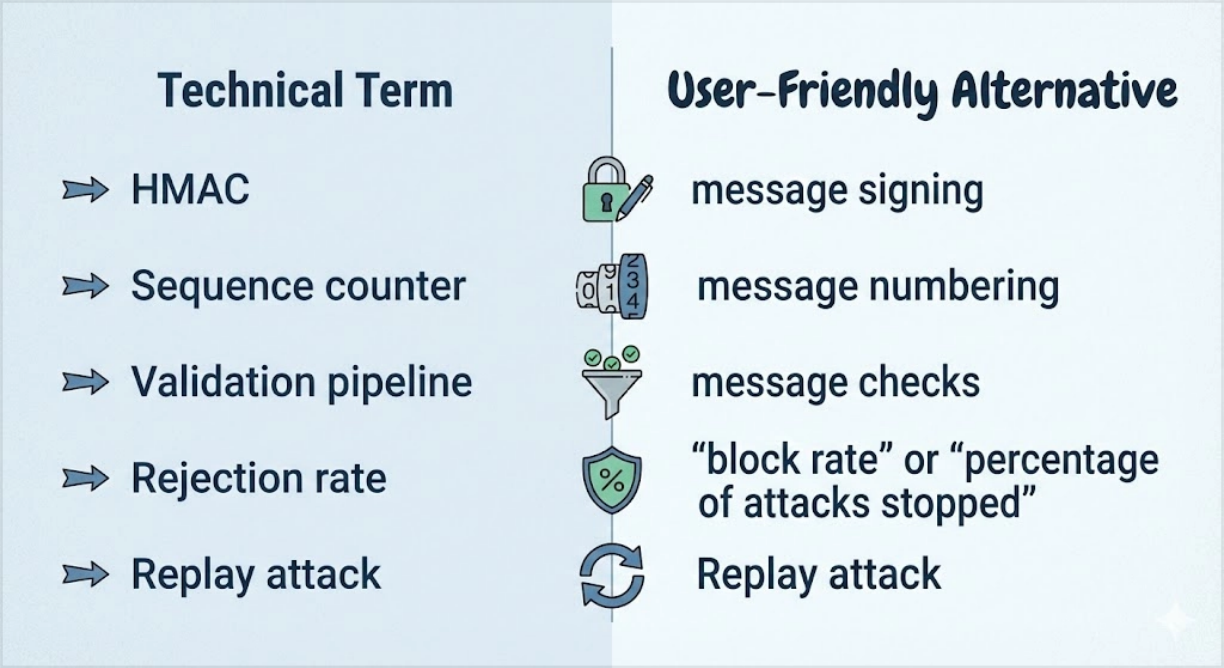
Structuring the Recommendation

A good recommendation has five components. Miss one and the reader has unanswered questions. Here's the structure.

1. The problem — one sentence describing the vulnerability.
2. What you tested — brief description of the experiments.
3. What you found — the key result.
4. Your recommendation — the specific action. This is the most important one. Don't bury the recommendation in qualifiers. State it clearly: deploy all three, here's why, here's the cost. If Marcus has to hunt for what you're actually suggesting, you've lost him.
5. What comes next — the forward look.

Writing for Non-Technical Stakeholders

Marcus doesn't know what HMAC stands for, and he doesn't need to. Neither does Sandra — she cares about alignment with standards, not implementation details. When you write the report, translate.



The best security recommendation is one that a hotel general manager can read, understand, and present to an auditor with confidence. If Marcus can explain it to Sandra without calling us over, we did our job. And don't forget — include that defense_comparison.png chart. A visual is worth more than a table of numbers when you're presenting to someone who doesn't think in rejection rates and sequence counters. That's the reality of this work. The people who approve budgets and sign off on deployments are rarely technical. Your job is to give them enough to make an informed decision, not to impress them with jargon.

Task: Defense Comparison Report

Task Link: https://docs.google.com/document/d/1X5XbuC2ytFvj6i9eOZWDWLESjn7YKNmPSrFZyjoSC98/edit?usp=sharing